DA2 – C++ MINI PROJECT

1) Smart Student Performance System – "Mentor Dashboard"

Scenario

You are asked to build a small "Mentor Dashboard" for a faculty mentor who tracks students' performance in 3 assessments (CAT1, CAT2, FAT). The mentor wants quick insights, not just marks.

Requirements

- Maintain student details: RegNo, Name, CAT1, CAT2, FAT
- Compute: Total, Average, Grade (A/B/C/Fail)
- Store records permanently in a file and reload on next run
- Allow:
    - Add new student
    - Search by RegNo
    - Update marks for a student
    - Delete a student record (only if mentor confirms)
- Reports:

1. **Topper(s)** by total
2. **Class statistics** (highest/lowest/average total)
3. **Grade distribution** (how many A, B, C, Fail)

- Validation:
    - Marks range 0–100
    - Duplicate RegNo not allowed

OOP Expectation

- class Student with methods like computeGrade(), display()
- class StudentManager manages vector<Student>, file read/write

2) Mini Banking System – "Personal Savings Account"

Scenario

A small cooperative society runs a simple savings account system. They don't use online banking, but they need a tool to track deposits/withdrawals with history.

Requirements

- Account: AccNo, Name, Balance (private)
- Operations:
  - Create new account (AccNo unique)
  - Deposit
  - Withdraw (no overdraft allowed)
  - Balance enquiry
- Transaction history:
  - Every deposit/withdraw must be logged with date/time OR a running transaction number
  - View last 5 transactions of an account
- File handling:
  - Store accounts file + transactions file and reload on startup
- Reports:
  - Total money held across all accounts
  - List accounts with balance below a threshold (e.g., < 500)

OOP Expectation

- class Account encapsulates balance + methods
- class Transaction
- class BankSystem handles vectors and file persistence

3) Inventory Management – "Shop Stock Controller"

Scenario

A small retail shop needs to track stock and sales. The owner wants alerts when stock goes low and wants to know the value of inventory.

Requirements

- Product: ProductID, Name, Price, Quantity
- Operations:
  - Add new product (ID unique)
  - Purchase stock (increase qty)
  - Sell stock (decrease qty; block if qty insufficient)
  - Search by ProductID / name keyword
  - Delete product (confirm before delete)
- Reports:
  - Low-stock report (below threshold)
  - Total inventory value = Σ(price × qty)
- File handling:
  - Save/load products list

OOP Expectation

- class Product
- class Inventory manages vector<Product>

4) Library Management – "Issue/Return with Rules"

Scenario

A department mini library issues books to students. A book cannot be issued to two students at the same time.

Requirements

- Book: BookID, Title, Author, Status (Available/Issued), IssuedToRegNo
- Operations:
  - Add book
  - Issue book (only if available)
  - Return book
  - Search by BookID OR title keyword
- Reports:
  - Show all issued books
  - Show count of available vs issued
- Validation:
  - Duplicate BookID not allowed
  - Return only if issued

OOP Expectation

- class Book controls status
- class Library manages vector<Book> + file save/load

5) Expense & Budget Tracker – "Monthly Money Coach"

Scenario

A working professional wants to track monthly expenses and avoid overspending.

Requirements

- Expense: Date (DD-MM), Category, Amount, Note
- Operations:
  - Add expense
  - View all expenses
  - Search by category / date
- Reports:
  - Monthly total
  - Category-wise total
  - Budget limit warning if monthly expense exceeds budget
- Validation:
  - Amount must be positive
  - Category must be from a fixed list OR validated non-empty

OOP Expectation

- class Expense
- class BudgetManager manages vector<Expense> + files

6) Attendance Manager – "Shortage Predictor"

Scenario

A class advisor wants to quickly identify shortage students based on attendance entries.

Requirements

- Student: RegNo, Name
- Attendance entry per day: P/A (store a simple record)
- Operations:
  - Add students list
  - Mark attendance for a date
  - View a student's attendance summary
- Reports:
  - Attendance % per student
  - List students below 75%
  - Class average attendance %
- Validation:
  - Marking attendance must accept only P/A
  - Duplicate student not allowed

OOP Expectation

- class Student
- class AttendanceManager uses vector and file persistence

7) Employee Payroll – "Salary Slip Generator"

Scenario

A small startup calculates monthly salary based on basic pay and overtime hours.

Requirements

- Employee: EmpID, Name, BasicPay, OT Hours
- Calculations:
  - Gross = Basic + (OT Hours × OT Rate)
  - Tax deduction slab (simple rule you define and document)
  - Net pay
- Operations:
  - Add employee
  - Update overtime
  - Generate payslip for an EmpID
- Reports:
  - Total salary payout for month
  - Highest paid employee
- Validation:
  - Salary and OT must be non-negative
  - EmpID unique

OOP Expectation

- class Employee does salary calculation methods
- class PayrollSystem manages vector<Employee> and file

8) Hospital Patient + Visit Tracker – "Clinic Register"

Scenario

A small clinic wants to maintain patient registration and visit notes.

Requirements

- Patient: PatientID, Name, Age, Phone
- Visit: PatientID, Date, Diagnosis, Prescription note
- Operations:
  - Register patient
  - Add visit record for a patient
  - Search patient and show visit history
- Reports:
  - Patients with more than N visits (frequent visitors)
  - Total visits recorded this month (basic)
- Validation:
  - PatientID unique
  - Cannot add visit for non-existing patient

OOP Expectation

- class Patient
- class Visit
- class HospitalSystem uses vector and file

9) Movie Ticket Booking – "Seat Locking System"

Scenario

A theatre has one screen. Seats once booked must not be booked again. Customers want receipt with booking ID.

Requirements

- Fixed seat layout: 5×10 = 50 seats
- Operations:
  - Display seat map (Available/Booked)
  - Book seats by seat numbers (validate range + availability)
  - Generate BookingID
  - Store booking with customer name, seat list, amount
  - Search booking by BookingID
- Reports:
  - Occupancy % (booked vs total)
  - Available seats count
- Validation:
  - Prevent duplicate seat selection in same booking
  - Prevent booking already booked seat

OOP Expectation

- class Show manages seat vector
- class Booking
- class TheatreSystem manages persistence

10) Bus Reservation + Cancellation – "Seat Release & Revenue"
Scenario
A travel agency runs 2–3 bus routes daily. They need booking + cancellation + revenue report.
Requirements
- Routes: RouteID, From, To, Fare, SeatCapacity (e.g., 30 seats)
- Operations:
    - Display available routes
    - Book ticket: passenger name + route + seat number(s)
    - Generate TicketID
    - Cancel ticket by TicketID (must release seats)
    - Search ticket by TicketID
- Reports:
    - Revenue per route
    - Total tickets booked per route
    - Most popular route (highest bookings)
- Validation:
    - Seat number must be within capacity
    - Seat cannot be double-booked
    - Cancel only if TicketID exists

OOP Expectation
- class Route with seat management
- class Ticket
- class ReservationSystem controls everything

Deliverables (What students must submit)

1. **Live hosted link** (GitHub Pages WASM preferred)
2. **PDF documentation** – Template shared

- Cover page with links
- Problem statement + features
- Struct + function list table
- File format description
- Testing table (min 6 test cases)
- Screenshots (min 3)
- AI usage declaration